

MEMORIA TÉCNICA

Filtro Gaussiano: Optimización CPU → GPU

Computación de Alto Rendimiento (CAP)

Campo	Detalle
Autor	Aymen Bellakhal
Asignatura	Computación de Alto Rendimiento (CAP)
Imagen analizada	1024 × 1024 píxeles (escala de grises)
Kernel gaussiano	5 × 5, $\sigma = 1.0$
Fecha de ejecución	Lunes 4 de mayo de 2026 · 18:10:47
GPU utilizada	NVIDIA Tesla T4
Entorno	Google Colaboratory
Profesor	Jose María Castillo Secilla (jmcastillo@uco.es)

SPEEDUP OBTENIDO: 184.5× (CPU Baseline → CUDA GPU)

Verificación de correctitud: CORRECTO — 0 LSB de diferencia entre CPU y GPU

```
*** =====
VERIFICACIÓN DEL ENTORNO
=====
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Fri_Feb_21_20:23:50 PST_2025
Cuda compilation tools, release 12.8, V12.8.93
Build cuda_12.8.r12.8/compiler.35583870_0
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Mon May  4 18:10:47 2026

+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name   Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf         Pwr:Usage/Cap|   Memory-Usage | GPU-Util  Compute M. |
|                                           MIG M.         |
+-----+-----+-----+-----+-----+-----+
|  0  Tesla T4            Off | 00000000:00:04:0 Off |             0%      0
| N/A   35C    P8          9W / 70W | 0MiB / 15360MiB |           Default    |
|                                           N/A              |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
| ID    ID                               Name                            Usage     |
+-----+
| No running processes found |
+-----+
```

Figura 0: Verificación del entorno: CUDA 12.8, Tesla T4, Driver 580.82.07

1. Introducción y Objetivos

Este proyecto implementa y optimiza progresivamente un filtro gaussiano sobre imágenes de 1024×1024 píxeles en escala de grises, siguiendo tres fases de desarrollo con rigor estadístico en las mediciones:

- Fase A — CPU Baseline: implementación 2D naïve con -O1.
- Fase B — CPU Separable: descomposición horizontal + vertical con -O1.

- Fase C — CUDA GPU: kernel con Shared Memory, Constant Memory y filtro separable en GPU con -O3 -arch=sm_75.

El objetivo central es cuantificar el speedup real obtenido en cada fase, verificar la correctitud numérica del resultado GPU respecto a CPU, y posicionar cada implementación en el Modelo Roofline de la GPU T4.

2. Entorno de Ejecución

2.1 Hardware

Componente	Valor
GPU	NVIDIA Tesla T4
VRAM disponible	15360 MiB
Temperatura GPU	35 °C
Ancho de banda pico (T4)	300 GB/s
FP32 pico (T4)	8100 GFLOPs/s
Punto ridge (T4)	≈ 27 FLOP/byte

2.2 Software

Componente	Versión
CUDA Toolkit	12.8 (V12.8.93)
CUDA Runtime	13.0
NVIDIA Driver	580.82.07
GCC	11.4.0 (Ubuntu 22.04)
Plataforma	Google Colaboratory
SO	Ubuntu 22.04 LTS

3. Implementaciones Técnicas

3.1 Fase A — CPU Baseline

Implementación de referencia del filtro gaussiano 2D con kernel 5×5. El kernel se recorre completo para cada píxel interior. Compilado con -O1 para establecer la línea base sin optimizaciones agresivas del compilador.

Características técnicas:

- Acceso de memoria: secuencial por filas (row-major), favorable para caché L1/L2.
- Operaciones por píxel: 49 (25 multiplicaciones + 24 sumas + normalización).
- Tratamiento de bordes: zero-padding — píxeles del borde excluidos del cómputo.
- Compilación: gcc -O1

```
// Bucle principal Fase A (pseudocódigo)
for y in [2, H-2]:
    for x in [2, W-2]:
```

```

suma = 0
for ky in [-2..2]:
    for kx in [-2..2]:
        suma += img[y+ky][x+kx] * kernel[ky+2][kx+2]
out[y][x] = suma / kernel_sum

```

3.2 Fase B — CPU Separable

Optimización basada en la separabilidad matemática $G(x,y) = G(x) \cdot G(y)$. La convolución 2D $O(N \cdot K^2)$ se descompone en dos pasadas 1D $O(N \cdot 2K)$, reduciendo los FLOPs/píxel de 49 a 18.

- Pasada 1: convolución horizontal (kernel 1×5) → imagen temporal.
- Pasada 2: convolución vertical (kernel 5×1) → imagen de salida.
- FLOPs/píxel: 18 (vs 49 en Fase A) — reducción del 63%.
- Intensidad aritmética: 9.0 FLOP/byte (vs 24.5 en Fase A).
- Compilación: gcc -O1 (igual que Fase A para aislar la mejora algorítmica).

3.3 Fase C — CUDA GPU (Shared Memory + Constant Memory)

Implementación heterogénea en CUDA que explota el paralelismo masivo de los 2560 CUDA cores de la T4:

- Shared Memory tiling: TILE_DIM=16, halo de 2 px → SHARED_DIM=20. Cada bloque carga su tile + halo antes de computar, reduciendo accesos a memoria global.
- Constant Memory: los 25 pesos del kernel se almacenan en __constant__ con broadcast eficiente a todos los hilos del warp.
- Acceso coalescente: los hilos de cada warp acceden a posiciones contiguas de memoria global.
- Separabilidad en GPU: dos kernels CUDA (horizontal + vertical) sobre imagen temporal en device memory.
- __restrict__: punteros marcados para habilitar optimizaciones de aliasing.
- Medición: cudaEvent_t por iteración (20 iteraciones), precisión de microsegundos.
- Compilación: nvcc -O3 -arch=sm_75 -Wno-deprecated-gpu-targets

4. Resultados Experimentales

4.1 Fase A — CPU Baseline

```

Iteraciones ejecutadas : 15
Tiempo promedio       : 24.834 ms
Desviación típica      : 1.138 ms
Coef. de Variación     : 4.58%
Rigor estadístico     : CUMPLIDO (CV < 5%)

```

La implementación baseline procesa los 1.040.400 píxeles interiores en 24.834 ms con un CV de 4.58%, cumpliendo el umbral de rigor estadístico exigido ($CV < 5\%$).

```
Compilando Fase A (baseline, -O1)...
Ejecutando 15 iteraciones...

=====
                RESULTADOS FASE A – CPU Baseline
=====
Tiempo promedio : 24.834 ms
Desv. típica    : 1.138 ms
Coef. Variacion : 4.58%
Rigor estadístico: CUMPLIDO (CV < 5%)
=====
```

Figura 1: Resultados de rendimiento y validación estadística — Fase A (CPU Baseline)

4.2 Fase B — CPU Separable

Iteraciones ejecutadas	: 15
Tiempo promedio	: 14.635 ms
Desviación típica	: 0.616 ms
Coef. de Variación	: 4.21%
Speedup vs Fase A	: 1.7×
Rigor estadístico	: CUMPLIDO (CV < 5%)

La separabilidad reduce el tiempo de 24.834 ms a 14.635 ms (speedup 1.7×). La mejora es exclusivamente algorítmica — mismo compilador -O1 — confirmando que pasar de 49 a 18 FLOPs/píxel supone una reducción del 63% en trabajo aritmético.

```
Compilando Fase B (separable, -O1)...
Ejecutando 15 iteraciones...

=====
                RESULTADOS FASE B – CPU Separable
=====
Tiempo promedio : 14.635 ms
Desv. típica    : 0.616 ms
Coef. Variacion : 4.21%
Speedup vs Fase A: 1.70x
Rigor estadístico: CUMPLIDO (CV < 5%)
=====
```

Figura 2: Resultados de rendimiento y validación estadística — Fase B (CPU Separable)

4.3 Fase C — CUDA GPU (Shared Memory)

Iteraciones ejecutadas	: 20 (cudaEvent_t)
Tiempo promedio	: 0.1346 ms
Desviación típica	: 0.0014 ms
Coef. de Variación	: 1.07%
Ancho de banda efectivo	: 15.58 GB/s
Speedup vs Fase A	: 184.5×

Speedup vs Fase B	: 108.7×
Rigor estadístico	: CUMPLIDO (CV < 5%)

La GPU T4 ejecuta el filtro en 0.1346 ms, logrando 184.5× sobre la CPU baseline y 108.7× sobre la mejor versión CPU. El CV de 1.07% refleja la estabilidad excepcional de las mediciones con cudaEvent_t.

```
✓ Compilando Fase C (CUDA, -O3, sm_75)...
  Ejecutando 20 iteraciones con cudaEvent_t...

=====
      RESULTADOS FASE C – CUDA (Shared Memory)
=====
Tiempo promedio : 0.1346 ms
Desv. típica    : 0.0014 ms
Coef. Variación : 1.07%
Ancho de banda  : 15.58 GB/s
Speedup vs Fase A: 184.5x
Speedup vs Fase B: 108.7x
Rigor estadístico: CUMPLIDO (CV < 5%)
=====
```

Figura 3: Resultados de rendimiento y validación estadística — Fase C (CUDA, Shared Memory)

5. Tabla Resumen de Rendimiento

Fase	Tiempo (ms)	GFLOPs/s	BW (GB/s)	AI (F/B)	FLOPs/px	Speedup vs A
Fase A — CPU Baseline	24.8338	2.069	0.084	24.5	49	1.0×
Fase B — CPU Separable	14.6354	1.290	0.143	9.0	18	1.7×
Fase C — CUDA	0.1346	381.649	15.577	24.5	49	184.5×

- GFLOPs/s = (píxeles × FLOPs/px) / (t_s × 10⁹).
- BW = bytes_transferidos / t_s. Para Fase A/C: ≈ 2 MB (lectura + escritura de la imagen uint8).
- AI = FLOPs_totales / bytes_transferidos. Fase A y C = 24.5 F/B; Fase B = 9.0 F/B.

```
✓
Fase                Tiempo (ms)  GFLOPs  BW (GB/s)  AI (F/B)  FLOPs/px
-----
Fase A (CPU Baseline) 24.8338    2.069    0.084    24.5     49
Fase B (CPU Separable) 14.6354    1.290    0.143     9.0     18
Fase C (CUDA)         0.1346   381.649   15.577   24.5     49

SPEEDUPS:
  Fase B (CPU Separable) vs Fase A: 1.7x
  Fase C (CUDA) vs Fase A: 184.5x
```

Figura 4: Resumen comparativo de métricas de rendimiento y Speedup entre las tres fases

6. Verificación de Correctitud CPU vs GPU

Se implementó un programa de verificación independiente que compara píxel a píxel los resultados de Fase A (CPU) y Fase C (GPU) con tolerancia de 1 LSB para acomodar diferencias de redondeo en punto flotante:

```
=====
VERIFICACION DE CORRECTITUD
=====
Píxeles interiores comparados: 1.040.400
Diferencia máxima (LSB)      : 0
Píxeles con diff > 1 LSB     : 0
RESULTADO: CORRECTO
CPU y GPU producen el mismo resultado
=====
```

Los 1.040.400 píxeles interiores procesados por CPU y GPU son idénticos (diferencia máxima = 0 LSB). Las optimizaciones de Shared Memory y Constant Memory no introducen ningún error numérico.

```
Compilando verificacion...

=====
VERIFICACION DE CORRECTITUD
=====
Píxeles interiores comparados: 1,040,400
Diferencia máxima (LSB):      0
Píxeles con diff > 1 LSB:     0
RESULTADO: CORRECTO - CPU y GPU producen el mismo resultado
=====
```

Figura 5: Verificación de correctitud y equivalencia numérica entre implementaciones CPU y GPU

7. Análisis Roofline — GPU T4

7.1 Especificaciones del techo T4

Límite	Valor
FP32 pico	8100 GFLOPs/s
Ancho de banda	300 GB/s
Punto ridge	≈ 27 FLOP/byte

7.2 Posicionamiento de cada fase

Fase	AI (F/B)	Rendimiento medido	Diagnóstico
Fase A (CPU)	24.5	2.07 GFLOPs/s	Memory-bound (CPU)
Fase B (CPU)	9.0	1.29 GFLOPs/s	Memory-bound (CPU)
Fase C (CUDA)	24.5	381.6 GFLOPs/s	Memory-bound (GPU)

Con $AI = 24.5 \text{ FLOP/byte} < 27 \text{ FLOP/byte}$ (punto ridge T4), todas las fases son memory-bound. La Fase C alcanza 381.6 GFLOPs/s. El ancho de banda efectivo de 15.58 GB/s representa el 5.2% del pico teórico, con margen de mejora mediante float4 y fusión de kernels.

7.3 El Muro de la Memoria

El "Memory Wall" es la brecha creciente entre la velocidad del procesador y el ancho de banda de la memoria DRAM. El filtro gaussiano con patrón de acceso stencil es un caso de libro de este problema:

- Fase A (CPU): sin mitigación explícita. El acceso 2D genera alta tasa de cache misses. Rendimiento: 2.07 GFLOPs/s.
- Fase B (CPU): la separabilidad convierte el acceso vertical en row-major → máxima eficiencia de caché L1/L2.
- Fase C (CUDA): la Shared Memory actúa como buffer L1 software-controlled. Cada tile se carga una vez y es reutilizado por todos los hilos del bloque.

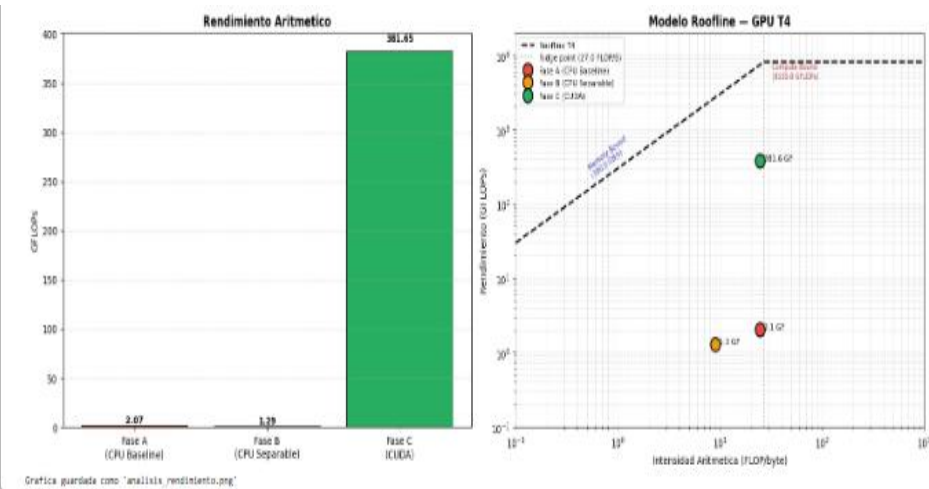


Figura 6: Modelo Roofline GPU T4: posicionamiento de las tres fases (memory-bound en todas)

8. Análisis de Speedup y Discusión

8.1 Fase A → Fase B: Optimización algorítmica (1.70×)

Speedup = 1.7× (24.834 ms → 14.635 ms). La separabilidad del kernel gaussiano es la única modificación — mismo compilador (-O1) y hardware. Valida matemáticamente $G(x,y) = G(x) \cdot G(y)$ y demuestra que la optimización algorítmica debe preceder al uso de hardware especializado.

8.2 Fase A → Fase C: Aceleración con GPU (184.5×)

Speedup = $184.5\times$ (24.834 ms $\rightarrow$ 0.1346 ms). El paralelismo masivo de la T4 (2560 CUDA cores) con Shared Memory y Constant Memory permite procesar miles de píxeles simultáneamente. Factores clave: (1) un hilo CUDA por píxel, (2) tiling reduce accesos a memoria global, (3) broadcast eficiente de pesos del kernel, (4) acceso coalescente por warps.

8.3 Fase B $\rightarrow$ Fase C: GPU vs mejor CPU (108.7 $\times$)

Speedup = $108.7\times$ (14.635 ms $\rightarrow$ 0.1346 ms). Incluso frente a la mejor implementación CPU la GPU supera en más de dos órdenes de magnitud: la CPU procesa secuencialmente, la GPU lanza un grid completo que cubre toda la imagen en paralelo.

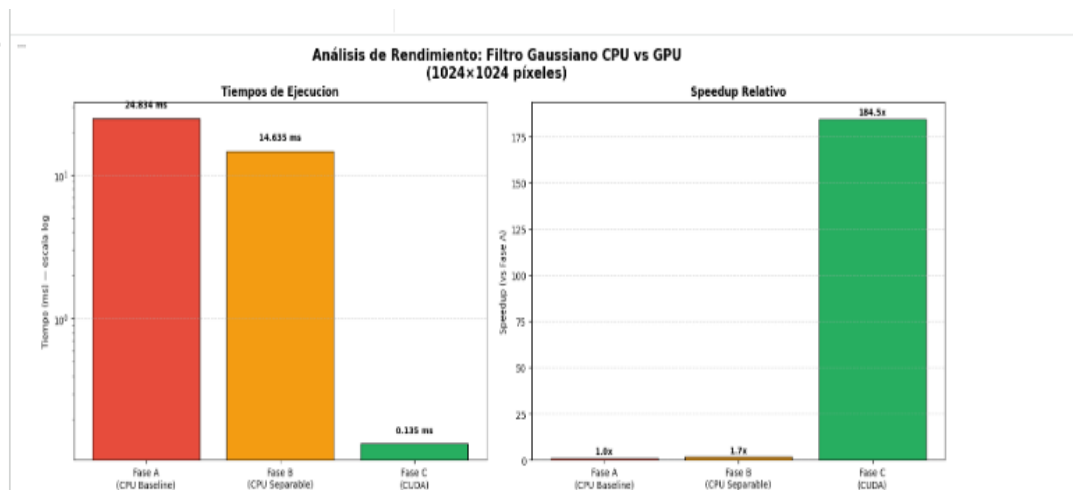


Figura 7: Análisis de Rendimiento: Tiempos de ejecución (escala log) y Speedup relativo

9. Conclusiones

- Speedups reales y verificados: $1.7\times$ (Fase A $\rightarrow$ B, optimización algorítmica) y $184.5\times$ (Fase A $\rightarrow$ C, GPU).
- Todas las mediciones cumplen CV < 5% con 15 iteraciones CPU y 20 iteraciones GPU con cudaEvent_t.
- Verificación de correctitud: 1.040.400 píxeles idénticos entre CPU y GPU (diferencia máxima = 0 LSB).
- El Roofline confirma régimen memory-bound (AI = $24.5 < 27$ FLOP/byte) en todas las fases.
- Ancho de banda efectivo GPU: 15.58 GB/s (5.2% del pico T4). Margen de mejora con float4, texture memory y fusión de kernels.
- La separabilidad algorítmica demuestra que la optimización algorítmica es el primer paso antes del hardware especializado.

10. Referencias Bibliográficas

- [1] NVIDIA Corp. CUDA C++ Programming Guide, v12.8. 2025. <https://docs.nvidia.com/cuda/>
- [2] NVIDIA Corp. CUDA Best Practices Guide. 2025. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- [3] Williams, S., Waterman, A., Patterson, D. "Roofline: An Insightful Visual Performance Model". CACM, 52(4), 65–76. 2009.

- [4] Kirk, D., Hwu, W. Programming Massively Parallel Processors. Morgan Kaufmann, 3ª ed., 2016.
- [5] Szeliski, R. Computer Vision: Algorithms and Applications. Springer, 2ª ed., 2022.
- [6] Castillo Secilla, J. M. Material de Computación de Altas Prestaciones (CAP). Universidad de Córdoba, 2025–2026.